

基於預訓練P-Code 表示與圖域適應之少樣本跨架構IoT 惡意軟體分類

林禹丞¹, 鄭欣明²

台灣科技大學資訊工程系智慧型系統實驗室^{1, 2}
{m11215206、smcheng}@mail.ntust.edu.tw^{1,2}

摘要

物聯網 (IoT) 裝置廣泛運行於異質指令集架構 (ISA)，使得現有惡意軟體分析方法難以跨平台通用。本文提出一種以Ghidra High P-Code 為統一中介表示之跨架構IoT 惡意軟體偵測框架，透過結合預訓練語言模型 (Pre-trained Language Model, PLM) 與圖注意力網路 (Graph Attention Network, GAT)，同時捕捉P-Code 指令序列之語意特徵與函式呼叫圖之結構特徵，並引入域適應 (Domain Adaptation) 機制，在目標架構僅有少量標記樣本的條件下縮小跨架構特徵分佈差距。實驗結果顯示，單架構場景下各架構F1-macro 均超過94%；跨架構場景下，直接遷移 (Lower Bound) 最低僅達0.19，引入域適應後 (Our Method) 提升至0.88 以上，驗證本框架之有效性。

關鍵字：跨架構惡意軟體分析、Ghidra P-Code、預訓練語言模型、圖神經網路、域適應

1. 前言

物聯網 (IoT) 生態系統正快速擴張，全球連網裝置數量於2025 年已達211 億[17]，廣泛部署於智慧家庭、工業控制、醫療照護等場域。不同於桌機或行動裝置集中於少數ISA，IoT 裝置橫跨x86、ARM、MIPS、PowerPC 等異質指令集架構 (ISA)，且普遍缺乏完善的安全防護，IoT 裝置遭受的攻擊事件從2018 年的3,270 萬次急升至2022 年的1.1229 億次[10]，資安威脅日益嚴峻[12]。

設計IoT 惡意軟體分析機制面臨兩項主要挑戰：第一，有別於傳統的PC 和行動裝置惡意軟體，IoT 惡意軟體的一項獨特特徵是能夠感染多種CPU 架構（包括ARM、MIPS、x86 等）的設備。因此，我們需要設計一種跨架構的分析方法，以偵測與分類IoT 惡意軟體[1]；第二，相同的惡意行為在不同ISA 下呈現截然不同的指令特徵，使得基於Byte Sequence 與Opcode 這些特徵的方法，針對單一架構所訓練的架構無法直接遷移。更現實的困難在於，惡意軟體樣本高度集中於x86-64 與ARM-32 等少數主流架構，其他低資源架構幾乎無標記資料可用。基於上述挑戰，有效的跨架構惡意軟體分析需要與架構無關的特徵，如Printable String, IR, 統計特徵。本研究針對跨架構惡意軟

體家族分析問題，以Ghidra High P-Code 作為特徵，以預訓練語言模型 (PLM) 對P-Code 指令序列進行深度語意編碼、以圖注意力網路 (GAT) 建模函式呼叫圖結構，並引入域適應 (Domain Adaptation) 機制，在目標架構僅有少量標記樣本的條件下縮小跨架構特徵分佈差距。

本文之主要貢獻如下：

- **架構無關特徵設計：**以High P-Code 之操作碼序列為語意輸入，捨棄含架構相依資訊之varnode 位址空間，並移除Thunk 與外部函式雜訊，提升跨架構IR 一致性；在此基礎上以x86-64 的資料從頭預訓練RoBERTa 捕捉P-Code 語意，並結合GAT 建模函式呼叫圖之拓撲結構。
- **少樣本域適應：**引入圖域適應機制，在目標架構僅有每類5 筆標記樣本的條件下，有效縮小跨架構特徵分佈差距，大幅提升遷移表現。
- **嚴格跨架構評估：**有別於現有研究[18, 2, 14]將多架構混合訓練的做法，本研究明確定義訓練僅來自單一Source 架構、測試來自另一Target 架構的嚴格場景，並在涵蓋ARM-32、Intel、MIPS、x86-64 之資料集上以10 組隨機種子驗證框架有效性。

2. 相關文獻

2.1 跨架構惡意軟體分析

在真正具備跨架構能力的特徵方面，可印字串 (Printable Strings) 因直接來自二進位檔案中的可讀字元序列，不依賴特定ISA 的指令集，天然具備架構無關性；Lee 等人與Cheng 等人[5, 3]均以可印字串作為IoT 惡意軟體分類特徵，驗證其在跨架構場景下的可遷移性。然而可印字串語意稀疏，且易受字串混淆攻擊影響。以平台無關中間表示 (IR) 為基礎的方法是另一類真正架構無關的特徵來源：CFDVex[11] 以VEX IR 結合CFG 特徵進行跨架構IoT 惡意軟體偵測；CIMALiR[13] 以IR 進行跨平台惡意軟體聚類；在跨架構指令表示學習方面，Redmond 等人[15]最早提出以NLP 方法學習跨架構指令嵌入，Unimap[20] 與Wang 等人[21]進一步以對比學習與低資源架構適應提升嵌入品質。

相對地，部分研究[18, 2, 14]雖標榜跨架構，但實驗設置為將多個架構資料混合後共同訓練與測試，本質上仍依賴Byte、Opcode 序列等架構相依特徵，訓練時已見過各架構樣本，並非以單一Source 架構訓練、另一Target 架構測試的場景進行實驗。此類設置無法反映真實世界環境所面臨的挑戰，其跨架構的表現存在高估，誤導了之後的跨架構惡意軟體分析研究。

2.2 中間表示與Ghidra P-Code

利用平台無關的中間表示 (IR) 是消除ISA 差異的主要策略之一。CFDVex[11] 以VEX IR 為基礎，結合CFG 特徵進行IoT 惡意軟體偵測。CIMALiR[13] 以IR 進行跨平台IoT 惡意軟體聚類。VulHawk[7] 則以IR 為基礎應用於跨架構函式相似性分析與漏洞偵測。我們先前的工作[6]以Ghidra P-Code 抽取14 維統計特徵，利用BinKit 2.0 資料集[4]驗證跨架構函式相似性，並以隨機森林進行分類，發現單架構準確率高達99.9%，但跨架構場景下性能嚴重衰退，揭示純統計特徵之語意對齊局限。

Ghidra[8] 為NSA 開發之開源逆向工程工具，其P-Code 是一種暫存器轉換語言 (Register Transfer Language, RTL)。相關文件與討論較為有限，目前較完整的研究為Naus 等人[9]針對P-Code 所進行的形式語意分析，並指出官方文件存在不完整與指令描述不夠精確等問題；其研究亦指出P-Code 並非設計來保證相同Source Code 在不同架構下必然產生完全相同的指令序列。

P-Code 分為Low P-Code 與High P-Code 兩層。Low P-Code 由SLEIGH (Ghidra 用於描述處理器指令集語意的DSL) 將機器碼直接展開，每條機器指令對應數條P-Code 操作，保留各架構暫存器名稱與低階副作用，具架構相依性。High P-Code 則由Ghidra 反編譯器在Low P-Code 之上進行資料流分析 (包含SSA 轉換、死碼消除、堆疊框架重建與變數識別) 後產生，消除架構特定的暫存器細節。本研究以High P-Code 作為特徵抽取的基礎。如圖 1 所示，對同一C 語言函式而言，MIPS、ARM 與x86-64 的High P-Code 幾乎一致，均收斂為相同的COPY 與RETURN 操作。

每條P-Code 指令由三個部分組成，其核心概念如下：

- **Varnode**：暫存器、記憶體位置與常數值的統一抽象，以三元組 $\langle space, offset, size \rangle$ 唯一識別。 $space$ 為具名位址空間 (ram、register、const、unique)， $offset$ 為該空間內的位元組索引， $size$ 為位元組數。varnode 本身不攜帶型別，型別由所在操作的語意隱式決定 (整數採二補數、布林為單位元組0/1、浮點依IEEE 754)。
- **Address Space**：P-Code 對記憶體與處理器資源的統一索引模型，定義為可定址的連續位元組序列。典型空間包含ram (主記憶體)、register (通用暫存器，偏移量由SLEIGH 靜態定義，具架構相依

表 1 P-Code 操作碼分類 (共71 個)

類別	操作碼
資料搬移	COPY, LOAD, STORE, PIECE, SUBPIECE
控制流程	BRANCH, CBRANCH, BRANCHIND, CALL, CALL/IND, RETURN
整數算術	INT_ADD/SUB/MULT/DIV/SDIV/REM/SREM/2COMP/NEGATE
整數比較	INT_EQUAL/NOTEQUAL/LESS/SLESS/LESSEQUAL/SLESSEQUAL/CARRY/SCARRY/SBORROW
整數位元	INT_AND/OR/XOR/LEFT/RIGHT/SRIGHT/ZEXT/SEXT/POPCOUNT/LZCOUNT
浮點運算	FLOAT_ADD/SUB/MULT/DIV/NEG/ABS/SQRT/CEIL/FLOOR/ROUND/EQUAL/NOTEQUAL/LESS/LESSEQUAL/NAN
布林與轉換	BOOL_NEGATE/XOR/AND/OR, INT2FLOAT, FLOAT2FLOAT, TRUNC, CALLOUTHER, CPOOL/REF, NEW
Additional	MULTIEQUAL, INDIRECT, PTRADD, PTRSUB, CAST, INSERT, EXTRACT

性)、const (立即數，偏移量本身即為數值) 與unique (臨時中間值)。

- **P-Code Operation**：定義操作類型，決定對varnode 執行的運算，操作的精度由輸入/輸出varnode 的大小決定，而非opcode 本身。P-Code 共定義71 個操作碼，其中Raw P-Code 64 個 (直接由機器指令轉譯)、Additional P-Code 7 個 (由後續分析引入，包含SSA ϕ -node MULTIEQUAL、間接副作用INDIRECT，以及指標與型別操作)，依功能分類如表 1 所示。

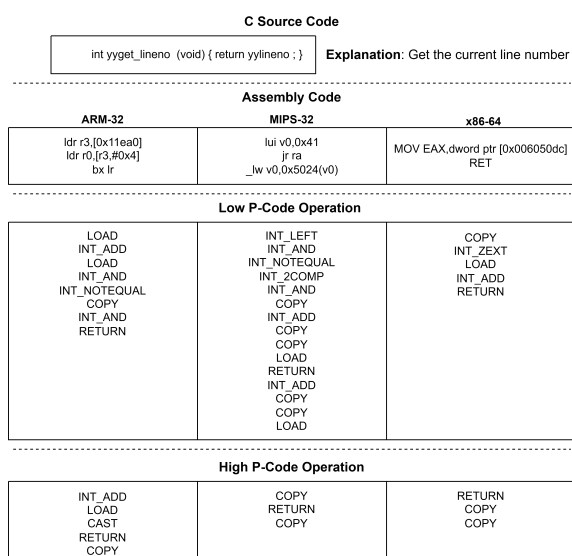


圖 1 同一C 函式在MIPS、ARM 與x86-64 架構下，從組合語言經Low P-Code 轉換為High P-Code 之過程。High P-Code 消除了架構間的語法差異，使語意等價函式趨於一致表示。

3. 方法

本節介紹所提方法之整體架構，包含P-Code 特徵表示、預訓練語言模型、圖注意力網路與域適應四個主要模組。圖 3 為整體框架概覽，圖 2 為特徵抽取流程。輸入為ELF 二進位檔，經Ghidra 反編譯產生High P-Code 後，抽取各函式之opcode 序列作為語意輸入（捨棄含架構相依資訊的varnode），並以函式呼叫關係建構呼叫圖。RoBERTa 對各節點的opcode 序列進行語意編碼，GAT 聚合呼叫圖拓撲結構，最終以域適應機制在少量目標域標記樣本的條件下對齊跨架構特徵分佈。

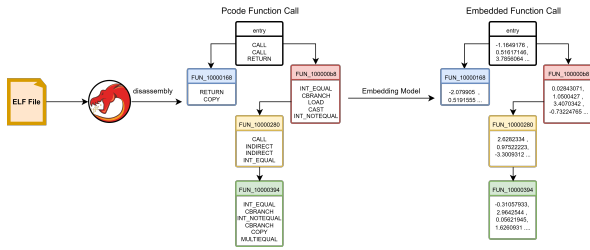


圖 2 特徵抽取流程：從ELF 二進位檔經Ghidra 轉換為High P-Code，再進行正規化與嵌入。

3.1 預訓練語言模型

本研究採用RoBERTa 預訓練語言模型對P-Code 指令序列進行語意編碼。選用RoBERTa 的關鍵原因在於P-Code 序列的長度特性：High P-Code 為函式級別的中間表示，一個函式往往包含數十至數百條操作，對應的token 序列遠長於一般程式語言的單行程式碼。如圖 4 所示，各架構下超過85% 的函式節點其P-Code token 長度不超過256，若將序列上限設為512 則可覆蓋絕大多數函式，保留完整的指令語意脈絡。相較於僅能處理較短序列的模型，RoBERTa 的Transformer 架構能透過自注意力機制捕捉整個函式內操作碼與運算元之間的長程依賴關係，有效建模P-Code 的語意結構。本研究以P-Code token 序列為輸入，在資料集上以遮罩語言模型（MLM）任務從頭預訓練，使模型學習P-Code 特有的語法與語意規律，最終以[CLS] token 的輸出作為函式節點的語意嵌入。

3.2 圖神經網路

將函式之控制流程圖（Control Flow Graph, CFG）轉換為圖結構輸入，透過圖神經網路（GNN）聚合鄰居節點資訊，捕捉程式之結構特徵。每個節點代表一個基本塊（Basic Block），節點特徵由預訓練語言模型之輸出提供。

3.3 對比式學習

為強化跨架構語意對齊，本研究引入對比式學習框架。將同一惡意軟體樣本於不同架構下之表示視

為正樣本對，不同樣本之表示視為負樣本對，透過對比損失函式縮短正樣本間之距離並拉大負樣本間之距離。

4. 實驗

本節系統性地評估所提框架在單架構與跨架構場景下的性能。我們以單架構訓練測試結果作為上界（Upper Bound），以直接跨架構遷移作為下界（Lower Bound），並與目標域少量標記資料之機器學習基準（Few-shot RF）進行比較，最終呈現引入域適應後之結果（Our Method）。為確保實驗結果之穩健性，除Lower Bound 外，所有設定均以10 組不同隨機種子重複實驗取均值與標準差，以消除少量樣本情境下樣本選取偏差的影響。

4.1 實驗設置

實驗環境為配備NVIDIA GeForce RTX 5070 Ti（顯存16,303 MiB）之工作站，作業系統為Ubuntu 24.04.3 LTS，CUDA 版本13.0。

預訓練語言模型（RoBERTa）：採用6 層Transformer 架構，隱藏維度256，8 個注意力頭，前饋層維度1024，最大序列長度512。以遮罩語言模型（MLM）任務預訓練，遮罩比例15%，Adam 優化器，學習率1e-4，Warmup 10,000 步，線性排程器，批次大小64，訓練100 個epoch。

圖注意力網路（GAT）：採用2 層架構，隱藏維度256，Dropout 0.2，注意力池化（Attention Pooling）。Adam 優化器，學習率0.001，批次大小32，最多訓練200 個epoch，Early Stopping（patience=15），損失函式為Cross Entropy。

4.2 資料集

本研究收集涵蓋ARM-32、ARM-64、Intel i386、MIPS、PowerPC（PPC）及AMD x86-64 共六種架構之IoT 惡意軟體樣本。所有樣本透過VirusTotal[19] 進行多引擎掃描，並以AVClass[16] 自動化工具統一標記惡意軟體家族，最終涵蓋十個家族，原始資料集共36,078 個樣本。

在前處理階段，我們首先移除加殼樣本，因打包技術會掩蓋惡意軟體的固有特徵並扭曲分類結果；其次移除重複樣本，以防止訓練與測試階段之間發生資料洩漏。此外，部分二進位檔因逆向工程失敗而被排除：逾時（超過600 秒）245 筆、節區格式錯誤1,432 筆、無邊1,017 筆、空節點72 筆，共計移除2,766 筆。最終有效資料集共33,312 個樣本，各家族與架構之詳細分布如表 2 所示。

4.3 實驗資料切割

為確保各實驗具備可比較性，本研究針對單架構與跨架構兩種場景分別制定資料切割策略。由

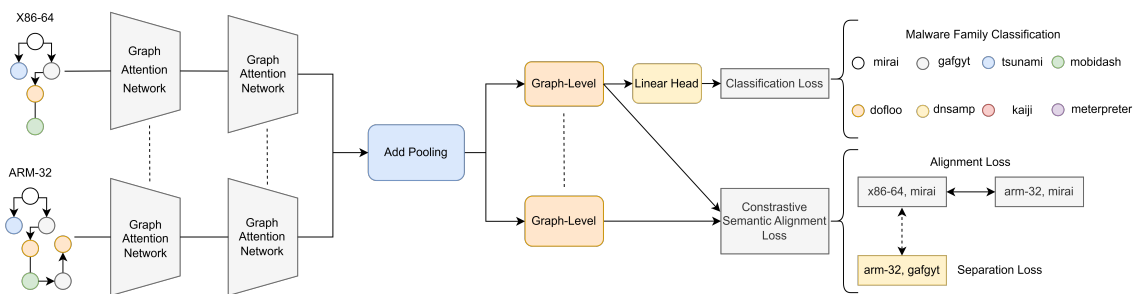


圖 3 所提跨架構惡意軟體偵測框架整體架構概覽。

表 2 實驗資料集各惡意軟體家族與架構之樣本數分布

Family	ARM-32	ARM-64	Intel	MIPS	PPC	x86_64	Total
mirai	1,018	331	1,083	1,651	3,218	1,185	8,486
gafgyt	1,054	20	1,094	1,715	3,214	1,203	8,300
tsunami	983	25	1,095	1,683	530	1,206	5,522
mobidash	1,063	284	1,098	306	0	1,216	3,967
dofloo	1,062	0	1,095	365	0	129	2,651
meterpreter	137	46	598	47	24	1,154	2,006
kaiji	134	91	211	164	0	244	844
wroba	467	49	284	0	0	2	802
dnsamp	95	0	99	64	0	208	466
fakebank	217	0	51	0	0	0	268
Total	6,230	846	6,708	5,995	6,986	6,547	33,312

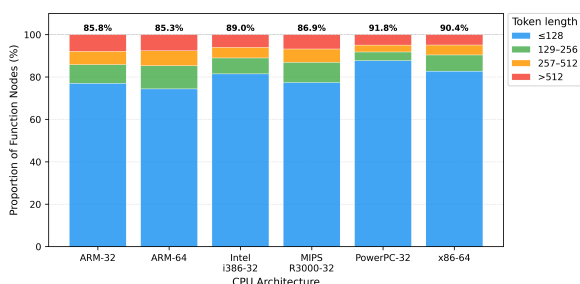


圖 4 各架構下函式節點P-Code token 長度之分布。粗體數字為max_length=256 時的覆盖率；設定max_length=512 可進一步涵蓋幾乎所有函式。

於IoT 真實環境所蒐集之樣本在各架構與各家族之間分布高度不均（如表 2），部分家族在特定架構下樣本數極少，直接隨機切割將導致測試集過小而評估不具代表性。為此，我們採用以下統一切割原則：測試集每個家族各取約50 個樣本（確保測試樣本數不低於50，以維持評量可靠性）；訓練集每個家族至少保留300 個樣本（若不足則取全部）。此一原則同時適用於單架構、跨架構與跨架構遷移學習三種實驗設置，確保三者採用相同的評量資料，具備公平可比較的基準。依此原則，實際參與實驗之八個家族及各架構樣本分布如表 3 所示，其中「~」表示樣本數低於300 而取全部。

表 3 跨架構實驗資料集各家族樣本數分布

Family	ARM-32	Intel	MIPS	x86_64
dnsamp	95	99	64	208
dofloo	300	300	300	129
gafgyt	300	300	300	300
kaiji	134	211	164	244
meterpreter	137	300	47	300
mirai	300	300	300	300
mobidash	300	300	300	300
tsunami	300	300	300	300

4.4 單架構分類

單架構實驗目的在於建立各架構下的性能上界，確認模型在同架構訓練與測試時的分類能力。預訓練語言模型（RoBERTa）以x86_64 架構資料進行預訓練；GAT 各自以當前架構之資料進行訓練與測試，採60/20/20 之訓練、驗證、測試比例分割，以10 種不同隨機種子重複實驗取均值與標準差。各架構均可達F1-macro 94% 以上，驗證所提框架在同架構場景下具備穩定之分類能力。

4.5 跨架構遷移學習比較

表 4 整合單架構與跨架構各設定之F1-macro，以統一評量指標進行比較，共四種設定：

Upper Bound（單架構）：Source 與Target 為同一架構，代表理想情境下的性能上界。以10 種隨機種子重複實驗，確保不同資料分割下結果穩

定。

Lower Bound (跨架構Source Only) : GAT以Source 架構全量資料訓練，直接在Target 架構測試，不進行任何適應，代表跨架構遷移之下界。

Few-shot Baseline (RF) : 不使用Source 資料，僅從Target 架構每個家族取5 筆樣本，對GNN Embedding 做Mean Pooling 後訓練Random Forest (200 棵樹，max_features=sqrt)。由於每類僅取5 筆，樣本選取結果對性能影響顯著，故以10 種隨機種子重複實驗取均值與標準差，確保評估結果不受特定樣本偏差影響。我們比較了XGBoost、SVM及Random Forest 多種模型與參數組合，Random Forest 表現最佳，故以此為代表。

Our Method (跨架構+ DA) : 在Source 架構訓練基礎上，引入對比語意對齊域適應框架，利用Target 架構每個家族5 筆有標記樣本進行對齊微調。同樣以10 種隨機種子重複實驗，消除少量樣本選取之隨機偏差。

Upper Bound 顯示各架構在同架構場景下均達 $F1 > 0.94$ ，確立模型能力之上界。Lower Bound 則揭示跨架構特徵分佈差距之嚴峻程度：以x86_64 訓練直接測試於MIPS 僅達0.2228，測試於ARM-32 為0.4659，顯示純深度語意特徵在跨架構場景下仍面臨顯著的分佈偏移問題。Few-shot Baseline 在無需Source 資料的條件下，僅以每類5 筆目標域樣本即可達 $F1$ 約0.80，反映GNN Embedding 所學得之語意表示本身已具備一定的跨架構遷移能力，可作為資料極度稀缺場景之有效基準。Our Method 結合Source 全量資料與目標域少量標記進行域適應，九個跨架構對之 $F1$ -macro 均達0.88 以上，最高為Intel $\rightarrow$ x86_64 之0.9491，較各對應Lower Bound 平均提升約0.29，亦全面超越Few-shot Baseline，驗證域適應框架在跨架構遷移場景下之顯著效益。

5. 結論

本文提出一種以Ghidra High P-Code 為統一中介表示之跨架構IoT 惡意軟體分析框架，結合RoBERTa 預訓練語言模型與圖注意力網路 (GAT)，同時建模P-Code 指令序列之語意特徵與函式呼叫圖之結構特徵。針對現實場景中目標架構標記資料匱乏的挑戰，本研究進一步引入域適應機制，僅需每類5 筆目標域樣本即可有效縮小跨架構特徵分佈差距。實驗結果顯示，單架構場景下各架構 $F1$ -macro 均超過94%；跨架構場景下，Our Method 相較於直接遷移之Lower Bound 最高提升0.66，驗證本框架之有效性。未來工作將補全所有架構對之域適應實驗，並探索更大規模預訓練模型與動態分析特徵之整合。

參考文獻

[1] M. Alhanahnah, Q. Lin, Q. Yan, N. Zhang, and Z. Chen. Efficient signature generation for classifying cross-architecture IoT malware. In

Proc. IEEE Conf. Communications and Network Security (CNS), pages 1–9, 2018.

[2] R. Chaganti, V. Ravi, and T. D. Pham. Deep learning based cross architecture internet of things malware detection and classification. *Computers & Security*, 120:102779, 2022.

[3] Y.-S. Cheng, T. Ban, S.-M. Cheng, T. Takahashi, and W.-C. Teng. Lightweight, architecture-agnostic IoT malware detection via printable strings. In *Proc. 2025 Workshop on Recent Advances in Resilient and Trustworthy MACHINE learning-driveN Systems*, pages 24–33, 2025.

[4] D. Kim et al. Revisiting binary code similarity analysis using interpretable feature engineering and lessons learned. *IEEE Transactions on Software Engineering*, 49(4):1661–1682, 2022.

[5] Y.-T. Lee et al. Cross platform IoT-malware family classification based on printable strings. In *Proc. IEEE 19th Int. Conf. Trust, Security and Privacy in Computing and Communications (TrustCom)*, pages 775–784, 2020.

[6] Y.-C. Lin et al. Cross-architecture IoT malware analysis using P-Code intermediate representation. In *Proc. Workshop on Recent Advances in Resilient and Trustworthy MACHINE learning-driveN Systems (RTMAN)*, pages 45–50, 2025.

[7] Z. Luo et al. VulHawk: Cross-architecture vulnerability detection with entropy-based binary code search. In *Proc. NDSS*, 2023.

[8] National Security Agency. Ghidra. [Online]. Available: <https://github.com/NationalSecurityAgency/ghidra>. Accessed: Feb. 2026.

[9] N. Naus et al. A formal semantics for P-Code. In *Working Conf. Verified Software: Theories, Tools, and Experiments*, pages 111–128. Springer, 2022.

[10] A. Petrosyan. Distribution of cyber-attacks across worldwide industries in 2023. Statista. [Online]. Available: <https://www.statista.com/statistics/1315805/cyber-attacks-top-industries-worldwide/>, 2024. Accessed: Feb. 2026.

[11] T. N. Phu et al. CFDVex: A novel feature extraction method for detecting cross-architecture IoT malware. In *Proc. 10th Int. Symp. Information and Communication Technology*, pages 248–254, 2019.

表 4 單架構與跨架構各設定F1-macro 比較 (mean $\pm$ std)

Architecture	F1-macro	
	Upper Bound	Few-shot RF
ARM-32	0.9613 $\pm$ 0.0095	0.7978 $\pm$ 0.0339
Intel	0.9781 $\pm$ 0.0049	0.8256 $\pm$ 0.0365
MIPS	0.9448 $\pm$ 0.0042	0.8010 $\pm$ 0.0375
x86_64	0.9653 $\pm$ 0.0154	0.8163 $\pm$ 0.0314
Source $\rightarrow$ Target	Lower Bound	Our Method
x86_64 $\rightarrow$ ARM-32	0.4659	0.9249 $\pm$ 0.0156
x86_64 $\rightarrow$ Intel	0.8483	0.9178 $\pm$ 0.0040
x86_64 $\rightarrow$ MIPS	0.2228	0.8806 $\pm$ 0.0288
ARM-32 $\rightarrow$ Intel	0.7800	0.8916 $\pm$ 0.0147
ARM-32 $\rightarrow$ MIPS	0.5271	0.9214 $\pm$ 0.0295
ARM-32 $\rightarrow$ x86_64	0.6723	0.8939 $\pm$ 0.0061
Intel $\rightarrow$ ARM-32	0.6993	0.9323 $\pm$ 0.0104
Intel $\rightarrow$ MIPS	0.4398	0.8915 $\pm$ 0.0133
Intel $\rightarrow$ x86_64	0.9185	0.9491 $\pm$ 0.0076

- [12] S. U. Qureshi, J. He, S. Tunio, N. Zhu, A. Nazir, A. Wajahat, F. Ullah, and A. Wadud. Systematic review of deep learning solutions for malware detection and forensic analysis in IoT. *Journal of King Saud University - Computer and Information Sciences*, page 102164, 2024.
- [13] M. A. Rahat et al. CIMALiR: Cross-platform IoT malware clustering using intermediate representation. In *Proc. IEEE 14th Annual Computing and Communication Workshop and Conference (CCWC)*, pages 0460–0466, 2024.
- [14] V. Ravi, T. D. Pham, and M. Alazab. Attention-based multidimensional deep learning approach for cross-architecture IoMT malware detection and classification in healthcare cyber-physical systems. *IEEE Transactions on Computational Social Systems*, 10(4):1597–1606, 2022.
- [15] K. Redmond, L. Luo, and Q. Zeng. A cross-architecture instruction embedding model for natural language processing-inspired binary code analysis. *arXiv preprint arXiv:1812.09652*, 2018.
- [16] M. Sebastián, R. Rivera, P. Kotzias, and J. Caballero. AVClass: A tool for massive malware labeling. In *Proc. Int. Symp. Research in Attacks, Intrusions, and Defenses (RAID)*, pages 230–253, 2016.
- [17] S. Sinha. State of IoT 2025: Number of connected IoT devices growing 14% to 21.1 billion globally. [Online]. Available: <https://iot-analytics.com/number-connected-iot-devices/>, 2024. Accessed: Feb. 2026.
- [18] D. Vasan, M. Alazab, S. Venkatraman, J. Akram, and Z. Qin. MTHAEL: Cross-architecture IoT malware detection based on neural network advanced ensemble learning. *IEEE Transactions on Computers*, 69(11):1654–1667, 2020.
- [19] VirusTotal. VirusTotal – free online virus, malware and url scanner. [Online]. Available: <https://www.virustotal.com>. Accessed: Mar. 2026.
- [20] J. Wang et al. Can a deep learning model for one architecture be used for others? Retargeted-Architecture binary code analysis. In *Proc. 32nd USENIX Security Symp.*, pages 7339–7356, 2023.
- [21] J. Wang, Q. Zeng, and L. Luo. Learning cross-architecture instruction embeddings for binary code analysis in low-resource architectures. In *Findings of the Association for Computational Linguistics: NAACL 2024*, pages 1320–1332, 2024.